

```

"""
LetzPlay Tennis Court Booking Automation
Reserva automaticamente quadras de tênis todo sábado às 9h e 10h.
Quadra 2 | Login com email e senha | Prioridade: 9h
"""

import asyncio
import logging
import os
import sys
from datetime import datetime, timedelta

from playwright.async_api import async_playwright, TimeoutError as PlaywrightTime

# — Configuração de logging —————
os.makedirs("logs", exist_ok=True)
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[
        logging.StreamHandler(sys.stdout),
        logging.FileHandler("logs/booking.log"),
    ],
)
log = logging.getLogger(__name__)

# — Configuração via variáveis de ambiente —————
EMAIL = os.environ["LETZPLAY_EMAIL"]
PASSWORD = os.environ["LETZPLAY_PASSWORD"]
COURT = os.environ.get("LETZPLAY_COURT", "2") # Quadra 2 por padrão
# 9h sempre vem primeiro — é a prioridade. 10h é tentada na sequência.
HORARIOS = os.environ.get("LETZPLAY_HORARIOS", "09:00,10:00").split(",")
BASE_URL = "https://letzplay.me"

# — Helpers —————
def proximo_sabado() -> datetime:
    """Retorna o próximo sábado (7 dias à frente quando rodamos na meia-noite de
    hoje = datetime.now())
    dias_ate_sabado = (5 - hoje.weekday()) % 7 # 5 = sábado
    if dias_ate_sabado == 0:
        dias_ate_sabado = 7 # já é sábado → pegar o seguinte
    return hoje + timedelta(days=dias_ate_sabado)

```

```

async def screenshot(page, nome: str):
    """Salva screenshot para debug."""
    caminho = f"logs/debug_{nome}_{datetime.now().strftime('%H%M%S')}.png"
    await page.screenshot(path=caminho)
    log.info(f"📸 Screenshot salvo: {caminho}")

# — Etapas de automação —
async def fazer_login(page):
    log.info("🔑 Iniciando login com email e senha...")
    await page.goto(f"{BASE_URL}/home", wait_until="networkidle")

    # Clica no botão de login / entrar
    try:
        await page.click("text=Entrar", timeout=8_000)
    except PlaywrightTimeout:
        await page.click("text=Login", timeout=8_000)

    await page.wait_for_load_state("networkidle")

    # Aguarda o formulário de email/senha (não social login)
    await page.wait_for_selector("input[type='email']", input[name='email'], time

    await page.fill("input[type='email']", input[name='email']", EMAIL)
    await page.fill("input[type='password']", input[name='password']", PASSWORD)
    await page.click("button[type='submit']")

    await page.wait_for_load_state("networkidle")
    await screenshot(page, "pos_login")

    if "login" in page.url.lower() or "signin" in page.url.lower():
        raise RuntimeError("❌ Falha no login – verifique EMAIL e PASSWORD")

    log.info("✅ Login bem-sucedido")

async def navegar_para_reservas(page):
    log.info("📄 Navegando para reservas...")

    for texto in ["Reservar", "Agendar", "Quadras", "Courts"]:
        try:
            await page.click(f"text={texto}", timeout=4_000)
            await page.wait_for_load_state("networkidle")
            log.info(f"✅ Navegou via '{texto}'")
            return
        except PlaywrightTimeout:

```

```

        continue

# Fallback: URL direta
await page.goto(f"{BASE_URL}/booking", wait_until="networkidle")
await screenshot(page, "pagina_reservas")

async def selecionar_data(page, data: datetime):
    """Navega o calendário dinâmico até a data-alvo."""
    log.info(f"📅 Selecionando data: {data.strftime('%d/%m/%Y')}")

    dia_str = str(data.day)
    mes_alvo = data.month

    # Navega meses se necessário (máx 3 tentativas)
    for _ in range(3):
        await page.wait_for_selector(
            ".calendar, [class*='calendar'], [class*='Calendar']", timeout=10_000
        )

        header = await page.query_selector(
            ".calendar-header, [class*='month'], [class*='Month']"
        )
        if header:
            header_text = (await header.inner_text()).lower()
            meses_pt = ["jan", "fev", "mar", "abr", "mai", "jun", "jul", "ago", "set", "out", "nov", "dez"]
            mes_encontrado = next(
                (i + 1 for i, m in enumerate(meses_pt) if m in header_text), None
            )
            if mes_encontrado == mes_alvo:
                break

        try:
            await page.click(
                "[aria-label*='próximo'], [aria-label*='next'], .calendar-next, button",
                timeout=4_000,
            )
            await asyncio.sleep(0.8)
        except PlaywrightTimeout:
            log.warning("Botão 'próximo mês' não encontrado — assumindo mês correto")
            break

    # Clica no dia
    seletores_dia = [
        f"td:has-text('{dia_str}'):not([class*='disabled']):not([class*='other'])",
        f"button:has-text('{dia_str}'):not([disabled])",
        f"[class*='day']:has-text('{dia_str}'):not([class*='disabled'])",
    ]

```

```

        f"span:has-text('{dia_str}'))",
    ]

    for sel in seletores_dia:
        try:
            await page.click(sel, timeout=4_000)
            log.info(f"✅ Dia {dia_str} selecionado")
            await asyncio.sleep(1)
            return
        except PlaywrightTimeout:
            continue

    await screenshot(page, "falha_selecionar_data")
    raise RuntimeError(f"❌ Não conseguiu clicar no dia {dia_str} do calendário")

async def selecionar_quadra(page):
    """Seleciona a Quadra 2 (ou o valor de COURT)."""
    log.info(f"🟡 Selecionando Quadra {COURT}...")

    seletores = [
        f"[class*='court']:has-text('{COURT}'))",
        f"[class*='quadra']:has-text('{COURT}'))",
        f"text=Quadra {COURT}",
        f"text=Court {COURT}",
    ]

    for sel in seletores:
        try:
            await page.click(sel, timeout=4_000)
            log.info(f"✅ Quadra {COURT} selecionada")
            return
        except PlaywrightTimeout:
            continue

    log.warning(f"⚠️ Quadra '{COURT}' não encontrada via seletor – pode já estar

async def selecionar_horario(page, horario: str) -> bool:
    """Seleciona o horário desejado. Retorna True se disponível."""
    log.info(f"🕒 Tentando reservar às {horario}...")

    try:
        await page.wait_for_selector(
            "[class*='slot'], [class*='time'], [class*='horario'], [class*='hour'
            timeout=10_000,
        )

```

```

except PlaywrightTimeout:
    log.warning("Seletores de horário genéricos não encontrados – tentando di

hora_variantes = [
    horario,
    horario.lstrip("0").replace(":", ":") if horario.startswith("0") else hor
    horario.replace(":", "h"),
]

for variante in hora_variantes:
    try:
        slot = await page.query_selector(
            f"[class*='slot']:has-text('{variante}'):not([class*='disabled'])"
        )
        if not slot:
            slot = await page.query_selector(f"text='{variante}'")

        if slot:
            await slot.click()
            log.info(f"✅ Horário {variante} selecionado")
            await asyncio.sleep(0.8)
            return True
    except Exception:
        continue

log.warning(f"⚠️ Horário {horario} não disponível ou não encontrado")
await screenshot(page, f"horario_nao_encontrado_{horario.replace(':', '')}")
return False

async def confirmar_reserva(page) -> bool:
    """Clica em confirmar / finalizar reserva."""
    botoes = ["Confirmar", "Reservar", "Finalizar", "Confirm", "Book"]

    for texto in botoes:
        try:
            await page.click(f"button:has-text('{texto}')" , timeout=5_000)
            await page.wait_for_load_state("networkidle")
            await asyncio.sleep(1.5)

            # Verifica mensagem de sucesso
            for sel in ["text=Reserva confirmada", "text=Reservado com sucesso",
                        "text=Booking confirmed", "[class*='success']", "[class*="
            if await page.query_selector(sel):
                log.info(f"✅ Reserva confirmada com sucesso!")
                await screenshot(page, "reserva_confirmada")
                return True

```

```

        log.info("Confirmação clicada (sem mensagem de sucesso explícita)")
        await screenshot(page, "pos_confirmacao")
        return True

    except PlaywrightTimeout:
        continue

    log.error("❌ Botão de confirmação não encontrado")
    await screenshot(page, "falha_confirmacao")
    return False

# — Fluxo por horário —————
async def reservar_horario(page, data: datetime, horario: str) -> bool:
    try:
        await navegar_para_reservas(page)
        await selecionar_data(page, data)
        await selecionar_quadra(page)

        disponivel = await selecionar_horario(page, horario)
        if not disponivel:
            return False

        return await confirmar_reserva(page)

    except Exception as e:
        log.error(f"❌ Erro ao reservar {horario}: {e}")
        await screenshot(page, f"erro_{horario.replace(':', ' ')}")
        return False

# — Main —————
async def main():
    log.info("=" * 60)
    log.info("🟢 LetzPlay Booking Bot iniciado")
    log.info(f"    Quadra          : {COURT}")
    log.info(f"    Horários         : {HORARIOS}    (9h = prioridade)")

    data_alvo = proximo_sabado()
    log.info(f"    Data alvo        : {data_alvo.strftime('%d/%m/%Y')} (sábado)")
    log.info("=" * 60)

    resultados: dict[str, str] = {}

    async with async_playwright() as p:
        browser = await p.chromium.launch(

```

```

        headless=True,
        args=["--no-sandbox", "--disable-setuid-sandbox"],
    )
    context = await browser.new_context(
        viewport={"width": 1280, "height": 900},
        user_agent=(
            "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) "
            "AppleWebKit/537.36 (KHTML, like Gecko) "
            "Chrome/120.0.0.0 Safari/537.36"
        ),
    )
    page = await context.new_page()

    try:
        await fazer_login(page)

        # HORARIOS já está ordenado com 9h primeiro (prioridade)
        for horario in HORARIOS:
            horario = horario.strip()
            log.info(f"\n— Reservando {horario} —")
            ok = await reservar_horario(page, data_alvo, horario)
            resultados[horario] = "✅ Reservado" if ok else "❌ Falhou"

            if not ok and horario == HORARIOS[0].strip():
                log.warning("⚡ 9h falhou - tentando 10h mesmo assim...")

            await asyncio.sleep(2)

        finally:
            await browser.close()

    # — Resumo final —————
    log.info("\n" + "=" * 60)
    log.info("🇧🇷 RESUMO DA EXECUÇÃO")
    for horario, status in resultados.items():
        prioridade = "← PRIORIDADE" if horario == HORARIOS[0].strip() else ""
        log.info(f"    {horario} → {status}{prioridade}")

    # Falha crítica apenas se o horário prioritário (9h) não foi reservado
    horario_prioritario = HORARIOS[0].strip()
    if "❌" in resultados.get(horario_prioritario, "❌"):
        log.error("🚨 Horário prioritário (9h) não reservado!")
        sys.exit(1)

    log.info("✅ Missão cumprida!")
    log.info("=" * 60)

```

```
if __name__ == "__main__":  
    asyncio.run(main())
```